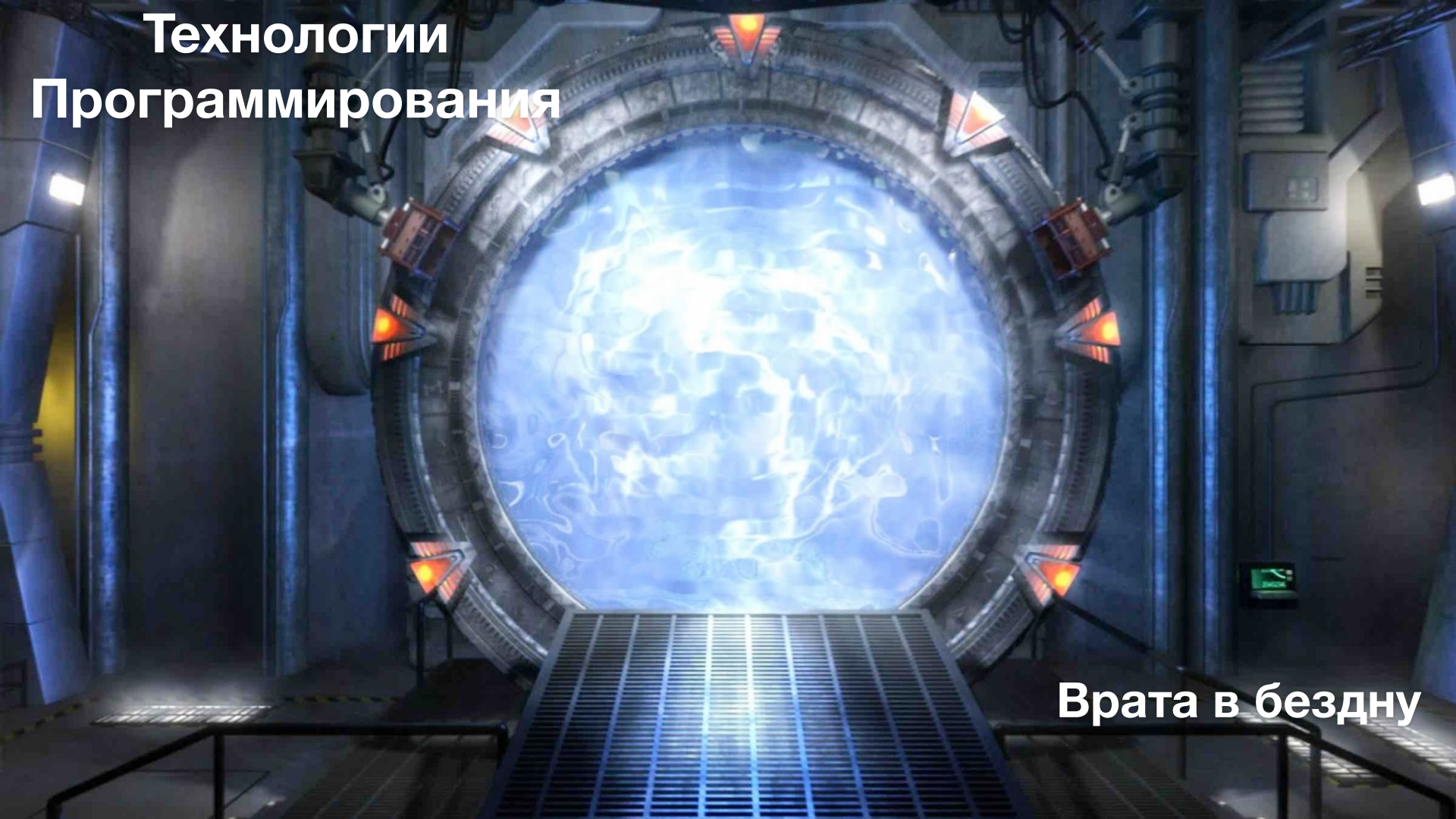


Технологии Программирования



Врата в бездну

Корабль Гоаулдов

```
struct HugeObject;  
sizeof(HugeObject) == 1MB;
```

- Структура передать в функцию

```
void DoSomethingWithObject(HugeObject object);
```

- Скорость передачи - 100MB/сек
- Вызов в цикле

```
for (int i = 0; i < 10000; ++i)  
{  
    DoSomethingWithObject(object);  
}
```



Всё ли в порядке с кодом, можно его исправить?

**Передача параметра в функцию по типу - это
ВСЕГДА КОПИРОВАНИЕ**

Шевроны

- **Указатель** - адрес переменной в памяти
 - Число
 - `size_t` (8B)
 - `type*`
- **Варианты указателя**

```
int* int_pointer = nullptr; // int_pointer - указатель на произвольную область памяти
int the_integer_array[512] = {0}; // the_integer_array - указатель на 0й элемент массива
```

- **Ошибки**
 - Невалидный указатель - не указывает ни на какой объект,
 - Обращение к освобожденному объекту,
 - Обращение к указателю на локальный объект.
- **Безопасность**

```
void* pointer = nullptr; // корректно. Самый корректный вариант
void* pointer = 0;       // корректно но не желательно. C-style. Pointer приводится к числу
void* pointer = NULL;    // Только только для Windows
```

Аналогия: номер телефона.

Можно позвонить не туда, по номеру который не обслуживается, итд.



Открытые врата



- **Ссылка на объект**

- Число (доступ к нему получить нельзя)
- Разыменованный указатель
- `size_t` (8B)

```
HugeObject& obj_ref = original_object; // Ссылка должна быть проинициализирована сразу
```

- В структуре поле может быть неинициализированной ссылкой.

```
struct SubObject
{
    HugeObject& objectRef; // инициализация объекта структуры обязательна: SubObject obj = {huge_object};
};
```

- **Ошибки**

- Обращение к ссылке на локальный объект за пределами его зоны видимости

- **Безопасность**

- Ссылку нельзя создать неинициализированной. Всегда требуется объект на который она ссылается

Аналогия - набранный номер, на который ответили

Можно сослаться на существующий объект, который удалится

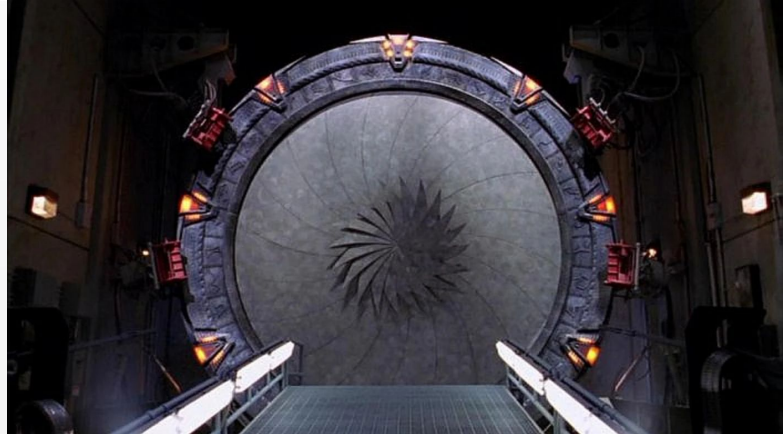
Безопасность превыше всего

- **Внимание**
 - Изменение объекта по указателю или ссылке меняет его для всех кто держит указатель или ссылку или сам объект
- **Ссылки как параметр в функции**

```
void functionRef(HugeObject& object);           // объект можно поменять внутри  
void functionConstRef(const HugeObject& object); // объект не меняется внутри функции
```

- **Указатели как параметр в функции**

```
void functionPtr(HugeObject* object);           // объект может быть не валидным. Объект можно поменять  
                                              // внутри функции  
void functionPtr(const HugeObject* object);     // Объект может быть не валидным. Объект нельзя поменять  
                                              // внутри функции
```



При работе с ссылками и указателями соблюдайте осторожность.

Вопрос!

Что будет?

```
uint8_t my_huge_data_array[10*1024*1024] = {0};
```

“Stack Overflow”

Кстати запомните - это очень полезный сайт.

База

- **Память выделенная под исполнение программы**
 - windows = 1MB но может быть изменена,
 - Linux $\leq$ 8MB.
- **Предназначена для хранения всех локальных переменных и функций в один момент времени.**
 - Чем больше функций вызвано в один момент времени, тем меньше места под переменные,
 - Каждая функция добавляет в стек свои данные,
 - Каждый jump (if),
 - Каждый цикл,
 - И так далее.



Стек кажется небольшим, но его достаточно для большинства задач

Карта вселенной

- **RAM**

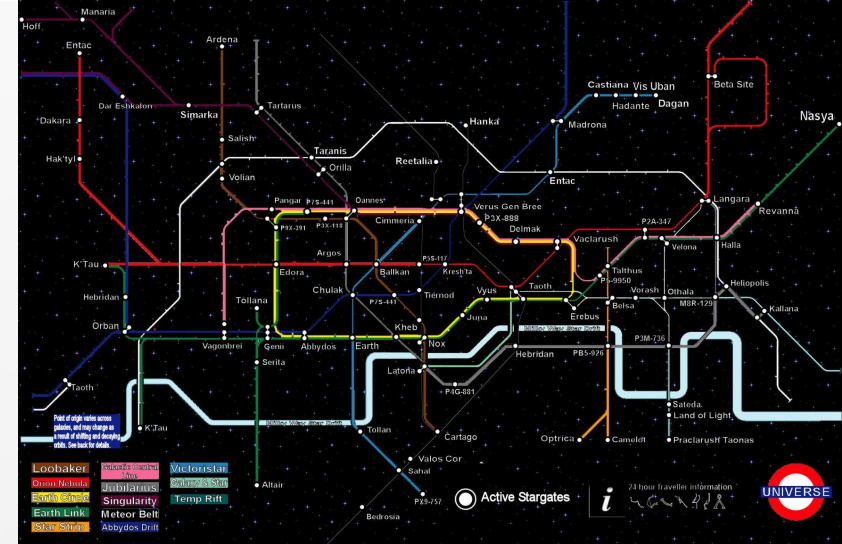
- Огромный объем памяти,
- Система резервирует под себя ~2GB,
- Всегда есть Swap.

- **Особенности работы с RAM**

- Рандомный доступ - адреса выделяются в произвольных местах,
- Положение аллокации не гарантируется:
 - Две последовательные аллокации не гарантируют последовательность адреса выделения.
- Гарантируется размер выделенного блока.

- **Структура указателя**

- Выделенная память,
- Сопутствующая информация - размер выделенной памяти, флаги доступа:
 - Пользовательского доступа к ней нет.



Путешествуем по старинке

- **Выделить память указанного размера**
 - malloc - выделение памяти в RAM в байтах,
 - calloc - выделение памяти в RAM в блоках.
- **Выделение нового блока с сохранением данных**
 - realloc - перевыделение памяти с сохранением содержимого.
- **Освобождение памяти**
 - free - освобождение памяти.



```
HugeObject* object = (HugeObject*)malloc(sizeof(HugeObject)); // выделили память
free(object); // освободили
```

Всегда освобождайте выделенную память
malloc - free

Вопрос!

```
struct Object
{
    int    IntegerField = 1;
    float FloatField = 2.f;
};

Object* my_object = (Object*)malloc(sizeof(Object)); // что будет в my_object.IntegerField?
```

А кто ж его знает...

malloc - отголосок старого C. Функция только выделяет память в RAM.
MALLOC НЕ ВЫЗЫВАЕТ КОНСТРУКТОР

Продвинутая техника

- **Выделить память**
 - `new` - выделяет единичный объект,
 - `new [ ]` - выделяет массив.
- **Выделение нового блока с сохранением данных**
 - N/A
- **Освобождение памяти**
 - `delete` - удаляет единичный объект,
 - `delete [ ]` - удаляет массив.

```
HugeObject* malloc_object = (HugeObject*)malloc(sizeof(HugeObject)); // просто выделит память
HugeObject* object = new HugeObject(); // вызовет конструктор, проинициализирует поля
int* array = new int [1024*1024]; // выделяет массив (!) содержимое будет произвольным
delete [ ] array; // очистили память занятую массивом
delete object; // освободили память занятую объектом
free(malloc_object);
```

Всегда освобождайте выделенную память соответствующей парой
`new - delete; new [ ] - delete [ ]`



Вопрос!

Как сделать realloc на C++?

```
int* realloc(int* array, size_t size, size_t new_size)
{
    int* new_array = new int [new_size];           // создаем временный массив старого размера
    memcpy(new_array, array, sizeof(int) * size); // копируем содержимое из старого массива в временный
    delete [] array;                               // удаляем старый массив
    return new_array;                              // возвращаем указатель на новый массив
}
```

Вопрос!

Что ж как сложно? А проще никак?

STL

Продвинутый флот



- **STL библиотека примитивов**
- **Индивидуальные указатели**
 - **unique_ptr** - указатель с уникальным владением. Освобождается при выходе unique_ptr из области видимости,
 - **shared_ptr** - указатель с разделенным владением. Освобождается когда все владельцы объекта выйдут из зоны видимости,
 - **weak_ptr** - не владеет объектом.
- **Контейнеры - в динамической памяти**
 - **array<type>** - массив фиксированного размера,
 - **vector<type>** - массив динамического размера, можно увеличивать и уменьшать по мере необходимости,
 - **string** - строка текста, массив char'ов (!) зависит от реализации,
 - **bitset** - массив битов - эквивалент vector<bool>.

И это далеко не всё...



FANDANGO
MOVIES

Добро пожаловать в бездну!